Contents

DevOps Master Cheatsheet — All Domains **1**
Linux Commands . . . 1
Git Commands . . . 2
Docker Commands . . . 3
Terraform Commands . . . 4
AWS CLI . . . 5
Helm Quick Reference . . . 6
Prometheus / PromQL Quick Reference . . . 6
Security Scanning Commands . . . 7
Quick Reference Cards . . . 8
HTTP Status Codes . . . 8
Port Quick Reference . . . 8
Git Commit Message Convention (Conventional Commits) . . . 8
File Permissions Quick Reference . . . 8

DevOps Master Cheatsheet — All Domains

Linux Commands

```
# PROCESS
ps aux                          # all processes
ps -ef --forest                 # tree view
pstree -p                       # visual tree
top / htop                      # live monitor
kill -SIGTERM PID               # graceful stop
kill -9 PID                     # force kill
strace -p PID                   # system call trace
lsof -p PID                     # files open by process
lsof -i :PORT                   # what's on a port
nice -n 10 CMD                  # lower priority
renice -n -5 -p PID             # change running priority

# PERFORMANCE
vmstat 1                        # memory, swap, cpu, io
iostat -xz 1                    # disk I/O per device
sar -r 1 10                     # memory history
mpstat -P ALL 1                 # per-CPU usage
perf top                        # CPU hotspots
free -m                         # memory in MB
uptime                          # load average (1/5/15 min)
df -h                           # disk usage
du -sh /path/*                  # directory sizes
dmesg | tail -50                # kernel messages

# FILES
find / -name "*.log" -mtime -1  # recent logs
find / -size +100M              # large files
lsof | grep deleted             # open deleted files (disk leak!)
stat file                       # inode info
ln -s TARGET LINK               # symbolic link
ln TARGET LINK                  # hard link

# NETWORKING
ss -tlnp                        # listening ports + PID
```

```
netstat -tulpn                    # older (deprecated)
tcpdump -i eth0 -n port 443       # capture traffic
tcpdump -w capture.pcap           # save capture
ip addr show                      # interfaces
ip route show                     # routing table
ip neigh show                     # ARP table
traceroute HOST                   # path tracing
mtr HOST                          # continuous traceroute
dig domain A                      # DNS lookup
dig +trace domain.com             # full DNS resolution trace
nslookup domain                   # simple DNS
curl -v https://url               # verbose HTTP
curl -I url                       # headers only
curl -o /dev/null -w "%{http_code}" url   # status code only

# SYSTEMD
systemctl status svc              # service status
systemctl start/stop/restart svc
systemctl enable/disable svc      # boot persistence
systemctl list-units --type=service --state=running
journalctl -u svc -f              # follow logs
journalctl -u svc --since "1 hour ago"
journalctl -p err -n 50           # last 50 errors
systemd-analyze blame             # slow boot services
```

Git Commands

```
# SETUP
git config --global user.name "Name"
git config --global user.email "email"
git config --global pull.rebase true      # prefer rebase on pull

# BASICS
git init / git clone URL
git add -p                                # interactive stage hunks
git commit -m "feat: add payment API"
git status / git log --oneline --graph

# BRANCHES
git branch -vv                            # branches + tracking
git checkout -b BRANCH                    # create + switch
git switch -c BRANCH                      # modern syntax
git push -u origin BRANCH                # push + track remote
git branch -d BRANCH                      # delete local
git push origin --delete BRANCH           # delete remote

# ADVANCED
git rebase -i HEAD~5                      # interactive rebase last 5
git rebase main                           # rebase onto main
git cherry-pick COMMIT_HASH               # apply specific commit
git cherry-pick A..B                      # range of commits
git bisect start; git bisect bad; git bisect good HASH  # bug search

# UNDO
git restore file                          # discard working changes
git restore --staged file                 # unstage
git reset --soft HEAD~1                  # undo commit, keep staged
```

```
git reset --mixed HEAD~1                # undo commit, keep working dir
git reset --hard HEAD~1                 # undo commit, discard changes
git revert COMMIT                        # safe undo (creates new commit)
git reflog                               # recover anything (90 days)

# STASH
git stash push -m "WIP: payment feature"
git stash list
git stash pop                            # apply + drop
git stash apply stash@{2}               # apply specific

# REMOTE
git fetch --prune                        # sync + remove stale remote branches
git remote -v                            # list remotes
git remote set-url origin NEW_URL

# INSPECT
git log --oneline --graph --all          # visual branch history
git diff HEAD~1..HEAD                    # last commit changes
git show COMMIT                          # commit details
git blame file                           # who changed each line
git log --follow -p -- file             # file history with diffs
git shortlog -sn                         # commits per author

# TAGS
git tag v1.2.3                           # lightweight tag
git tag -a v1.2.3 -m "Release v1.2.3" # annotated
git push origin --tags                   # push all tags
git push origin v1.2.3                  # push specific tag
```

Docker Commands

```
# IMAGES
docker build -t myapp:1.0.0 .
docker build --target production -t myapp:1.0.0 .     # multi-stage target
docker build --no-cache -t myapp:1.0.0 .
docker images                                          # list images
docker rmi IMAGE_ID                                    # remove
docker image prune -a                                  # remove unused
docker inspect IMAGE                                   # metadata

# CONTAINERS
docker run -d --name myapp -p 8080:8080 myapp:1.0.0
docker run --rm -it myapp:1.0.0 bash                 # interactive, auto-delete
docker run --env-file .env myapp:1.0.0
docker run -v $(pwd):/app -w /app myapp:1.0.0
docker run --memory="512m" --cpus="0.5" myapp:1.0.0
docker run --read-only --tmpfs /tmp myapp:1.0.0       # security hardened
docker run --user 1001:1001 myapp:1.0.0
docker ps -a                                           # all containers
docker stop / start / restart NAME
docker rm -f NAME                                      # force remove

# LOGS & DEBUG
docker logs -f CONTAINER                               # stream logs
docker logs --tail=100 CONTAINER
docker exec -it CONTAINER bash                         # shell
```

```
docker inspect CONTAINER                                    # full details
docker stats                                                # live resource usage
docker top CONTAINER                                        # processes inside
docker diff CONTAINER                                       # filesystem changes

# REGISTRY
docker login REGISTRY
docker tag myapp:1.0.0 ghcr.io/org/myapp:1.0.0
docker push ghcr.io/org/myapp:1.0.0
docker pull IMAGE
docker save IMAGE > image.tar                               # export
docker load < image.tar                                     # import

# VOLUMES & NETWORK
docker volume ls / create NAME / rm NAME / inspect NAME
docker network ls / create NAME / inspect NAME
docker network connect NETWORK CONTAINER

# CLEANUP
docker system prune -af                                     # remove all unused
docker system df                                            # disk usage
docker builder prune                                        # clear build cache

# COMPOSE
docker compose up -d --build
docker compose down -v                                      # remove volumes too
docker compose ps / logs -f / top
docker compose exec SERVICE bash
docker compose config                                       # validate
docker compose pull                                         # pull images
```

Terraform Commands

```
# INIT
terraform init                          # initialize + download providers
terraform init -upgrade                 # upgrade providers
terraform init -reconfigure             # change backend

# PLAN & APPLY
terraform plan                          # show what will change
terraform plan -out=tfplan              # save plan
terraform plan -target=resource.name    # target specific resource
terraform plan -var="env=production"
terraform plan -var-file="prod.tfvars"
terraform apply tfplan                  # apply saved plan
terraform apply -auto-approve           # skip confirmation (CI only!)
terraform destroy -target=resource.name

# STATE
terraform state list                    # list all resources in state
terraform state show resource.name      # show resource details
terraform state mv OLD NEW              # rename (avoid destroy/recreate)
terraform state rm resource.name        # remove from state (keep real resource)
terraform state pull                    # download remote state
terraform import resource.name REAL_ID  # import existing infra
terraform force-unlock LOCK_ID          # release stuck lock (careful!)
```

```
# VALIDATE & FORMAT
terraform fmt -recursive              # format all .tf files
terraform fmt -check                  # check formatting (CI)
terraform validate                    # syntax + logic check

# WORKSPACE
terraform workspace list
terraform workspace new staging
terraform workspace select production

# OUTPUT
terraform output                       # show all outputs
terraform output -json                 # JSON format
terraform output output_name           # specific output
terraform refresh                      # sync state with real infra

# GRAPH
terraform graph | dot -Tsvg > graph.svg  # visualize dependency graph
```

AWS CLI

```
# CONFIG
aws configure                          # set credentials
aws configure --profile prod           # named profile
export AWS_PROFILE=prod                # use profile
export AWS_DEFAULT_REGION=us-east-1

# EC2
aws ec2 describe-instances --filters "Name=instance-state-name,Values=running"
aws ec2 describe-instances --query 'Reservations[*].Instances[*].[InstanceId,PrivateIpAddress,Sta
aws ec2 stop-instances --instance-ids i-1234567890abcdef0
aws ec2 start-instances --instance-ids i-1234567890abcdef0

# S3
aws s3 ls s3://bucket-name/
aws s3 cp file.txt s3://bucket/path/
aws s3 sync ./local-dir s3://bucket/path/ --delete
aws s3 rb s3://bucket --force             # remove bucket + contents
aws s3api get-object-acl --bucket NAME --key KEY

# IAM
aws iam list-users
aws iam list-roles --query 'Roles[*].[RoleName,CreateDate]' --output table
aws iam get-role --role-name ROLE_NAME
aws sts get-caller-identity              # who am I?
aws sts assume-role --role-arn ARN --role-session-name SESSION

# SECRETS MANAGER
aws secretsmanager get-secret-value --secret-id NAME --query SecretString --output text
aws secretsmanager list-secrets --query 'SecretList[*].[Name,LastChangedDate]'

# EKS
aws eks update-kubeconfig --name CLUSTER_NAME --region REGION
aws eks list-clusters
aws eks describe-cluster --name CLUSTER_NAME --query 'cluster.[status,version,endpoint]'

# LOGS
```

```
aws logs describe-log-groups
aws logs tail /aws/eks/cluster/cluster --follow
aws logs filter-log-events --log-group-name NAME --filter-pattern "ERROR"

# COST
aws ce get-cost-and-usage \
  --time-period Start=2026-04-01,End=2026-04-30 \
  --granularity MONTHLY \
  --metrics BlendedCost \
  --group-by Type=SERVICE,Key=SERVICE
```

Helm Quick Reference

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo bitnami/postgresql --versions
helm show values bitnami/postgresql

helm install my-postgres bitnami/postgresql \
  --namespace databases --create-namespace \
  -f postgres-values.yaml \
  --set auth.postgresPassword=PASS

helm upgrade my-postgres bitnami/postgresql -f values.yaml
helm upgrade --install my-postgres bitnami/postgresql  # idempotent

helm list -A
helm status my-postgres -n databases
helm history my-postgres -n databases
helm rollback my-postgres 2 -n databases
helm uninstall my-postgres -n databases

helm template my-postgres bitnami/postgresql -f values.yaml  # render
helm get values my-postgres -n databases                      # installed values
helm get manifest my-postgres -n databases                    # rendered manifests
```

Prometheus / PromQL Quick Reference

```
# Rate (counter) — requests per second
rate(http_requests_total{job="api"}[5m])

# Error percentage
rate(http_requests_total{status=~"5.."}[5m]) /
rate(http_requests_total[5m]) * 100

# P99 latency from histogram
histogram_quantile(0.99,
  sum(rate(http_request_duration_seconds_bucket[5m])) by (le, service)
)

# Memory usage %
container_memory_working_set_bytes /
container_spec_memory_limit_bytes * 100
```

```
# CPU throttling %
rate(container_cpu_cfs_throttled_seconds_total[5m]) /
rate(container_cpu_cfs_periods_total[5m]) * 100

# Pods not ready
kube_deployment_spec_replicas - kube_deployment_status_ready_replicas

# Disk fill prediction (4 hours)
predict_linear(node_filesystem_free_bytes[2h], 4 * 3600) < 0

# Aggregations
sum by (service) (rate(requests_total[5m]))   # per service
avg without (instance) (cpu_usage[5m])        # collapse instance label
topk(5, memory_bytes)                          # top 5
```

Security Scanning Commands

```
# Container image scanning
trivy image myapp:latest
trivy image --severity HIGH,CRITICAL myapp:latest
trivy image --exit-code 1 --severity CRITICAL myapp:latest

# Filesystem scan (dependencies, IaC, secrets)
trivy fs --security-checks vuln,config,secret ./
grype myapp:latest

# SBOM generation
syft myapp:latest -o spdx-json > sbom.json
syft myapp:latest -o cyclonedx-json > sbom-cdx.json
grype sbom:./sbom.json

# Image signing
cosign sign --yes registry.io/org/myapp:v1.0.0
cosign verify --certificate-identity=CI@COMPANY.com \
  --certificate-oidc-issuer=https://token.actions.githubusercontent.com \
  registry.io/org/myapp:v1.0.0

# IaC scanning
checkov -d ./terraform --compact
tfsec ./terraform
kube-score score deployment.yaml
kubesec scan pod.yaml

# Secret scanning
gitleaks detect --source=. --verbose
detect-secrets scan . > .secrets.baseline
detect-secrets audit .secrets.baseline

# K8s security audit
kube-bench run --targets master,node
kubectl auth can-i --list --as=system:serviceaccount:default:myapp
```

Quick Reference Cards

HTTP Status Codes

```
2xx Success:  200 OK, 201 Created, 204 No Content
3xx Redirect: 301 Permanent, 302 Temp, 304 Not Modified
4xx Client:  400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx Server:  500 Internal Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout
```

Port Quick Reference

```
22    SSH
25    SMTP
53    DNS (TCP+UDP)
80    HTTP
443   HTTPS
2379  etcd client
2380  etcd peer
6443  Kubernetes API
8080  HTTP alt
9090  Prometheus
9093  Alertmanager
3000  Grafana
3100  Loki
5432  PostgreSQL
3306  MySQL
6379  Redis
5672  RabbitMQ AMQP
9092  Kafka
2181  ZooKeeper
```

Git Commit Message Convention (Conventional Commits)

```
feat:     new feature
fix:      bug fix
docs:     documentation
style:    formatting (no logic change)
refactor: code restructure (no feature/fix)
perf:     performance improvement
test:     adding tests
build:    build system changes
ci:       CI config changes
chore:    maintenance tasks
revert:   revert previous commit

Examples:
feat(auth): add JWT refresh token rotation
fix(api): prevent double-charge on timeout retry
feat!: drop support for Python 3.8  (! = breaking change)
```

File Permissions Quick Reference

```
chmod 755 → rwxr-xr-x (owner all, group/other read+exec)
chmod 644 → rw-r--r--  (owner read+write, group/other read)
chmod 600 → rw-------  (owner only, use for SSH keys)
chmod 777 → rwxrwxrwx  (NEVER in production)
chmod +x file           (add execute for all)
chmod u+s file          (SUID — run as owner)
chmod g+s dir           (SGID — new files inherit group)
chmod +t dir            (sticky — only owner can delete)
```